

Ball Machine

Մարինեն գնդակներով մեքենա է հնարել՝ IOI-ի մասնակիցներին գվարճացնելու համար: Մեքենայի ներքին կառուցվածքը հետևյալն է.

- Մեքենան բաղկացած է N **գագաթներից**, որոնք համարակալված են 0 -ից մինչև $N - 1$: $N - 1$ համարով գագաթը կոչվում է **արմատ**:
- Յուրաքանչյուր u գագաթի համար, որտեղ $0 \leq u < N - 1$, u -ի **ծնողը** $P[u]$ գագաթն է, որտեղ $P[u] > u$, իսկ u գագաթը $P[u]$ -ի **գավակ** է: Արմատը ծնող չունի: Չավակներ չունեցող գագաթը կոչվում է **տերև**:

Դուք **չգիտեք** N թիվը կամ որևէ u գագաթի $P[u]$ ծնողին: Դրա փոխարեն, Մարինեն ձեզ ասում է M թիվը՝ տերևների քանակը, և որ տերևների ինդեքսներն են՝ $0, 1, \dots, M - 1$: Ձեր խնդիրն է գտնել մեքենայի ներքին կառուցվածքը՝ այն գործարկելու միջոցով:

Մեքենայի յուրաքանչյուր գագաթ կարող է պարունակել առավելագույնը մեկ **գնդակ**, և յուրաքանչյուր գնդակ ունի **արժեք**, որը ոչ բացասական ամբողջ թիվ է: Մենք ասում ենք, որ գագաթն ունի x արժեք, եթե այն պարունակում է x արժեքով գնդակ: Գագաթը համարվում է **գբաղեցված**, եթե այն պարունակում է գնդակ, հակառակ դեպքում այն **ազատ** է: Սկզբնապես բոլոր գագաթները ազատ են:

Դուք կարող եք կատարել երկու տեսակի գործողություններ.

- **insert(U, X)**: գրեք X արժեքը նոր գնդակի վրա և փորձեք տեղադրել այն U տերևի վրա:
 - Եթե U տերևը զբաղված է, գործողությունը վերադարձնում է `false` առանց գնդիկը տեղադրելու:
 - Եթե U տերևը ազատ է, գնդակը տեղադրվում է U -ի վրա: Այնուհետև գնդակը շարժվում է դեպի իր ընթացիկ գագաթի ծնողը, քանի դեռ այդ ծնողը ազատ է: Շարժումը դադարում է, երբ գնդակը հասնում է արմատին կամ այնպիսի հանգույցի, որի ծնողը զբաղված է: Գործողությունը վերադարձնում է `true`:
- **collect()**: եթե արմատը ազատ է (ինչը նշանակում է, որ մեքենան դատարկ է), `collect` վերադարձնում է դատարկ զանգված: Հակառակ դեպքում, ստորև նկարագրված `traverse` ռեկուրսիվ ֆունկցիան հավաքում է մեքենայում ներկայումս գտնվող բոլոր գնդակների արժեքները S զանգվածի մեջ: Սկզբում S զանգվածը դատարկ է, և կանչվում է `traverse(N - 1)`:

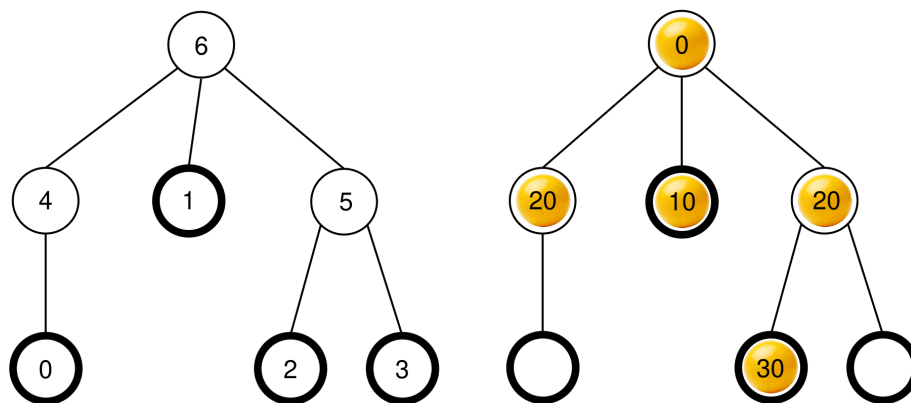
```

traverse(u):
    append the value of the ball at node u to the end of S
    let c[u] be the list of the occupied children of u
    sort c[u] in non-decreasing order according to the values
      of the balls at those nodes (if multiple nodes have the
      same value, they can appear in any order)
    for each v in c[u]:
        traverse(v)

```

Այս ֆունկցիայի ավարտից հետո բոլոր գնդակները հանվում են մեքենայից, և collect վերադարձնում է S :

Օրինակ, դիտարկենք $N = 7$ գազաթներով և $P = [4, 6, 5, 5, 6, 6]$ ծնողների գանգվածով մեքենան: Ձախ կողմի նկարը ցույց է տալիս դատարկ մեքենան՝ գազաթների համարներով, իսկ աջ կողմի նկարը ցույց է տալիս գնդակների հնարավոր կոնֆիգուրացիան որոշակի insert գործողությունների հաջորդականությունից հետո (այս հաջորդականությունը տրված է Օրինակ բաժնում):



Երբ կանչվում է `collect()` ֆունկցիան, արմատը (6 համարով գազաթը) զբաղված է, ուստի S -ը սկզբնաբաժանվում է որպես $S = []$ և կանչվում է `traverse(6)` ֆունկցիան:

traverse(6) : 6 գազաթն ունի 0 արժեք; այն S -ին կցուցնուց հետո $S = [0]$: 6 գազաթի գանգակներն են 4 , 1 և 5 գազաթները, որոնք բոլորն էլ զբաղված են: Դրանց արժեքներն են համապատասխանաբար 20 , 10 և 20 : Ոչ նվազող կարգով սորտավորումից ստացվում է $c[6] = [1, 5, 4]$ (նշենք, որ $c[6] = [1, 4, 5]$ նույնպես ճիշտ կլինի, քանի որ 4 և 5 գազաթներն ունեն նույն արժեքը): Այնուհետև ֆունկցիան կանչում է `traverse(c[6]` -ի յուրաքանչյուր գազաթի վրա այդ հերթականությամբ:

- **traverse(1)** : 1 համարով գազաթն ունի 10 արժեք; այն S -ին կցելուց հետո $S = [0, 10]$: 1 գազաթը զբաղված գանգակներ չունի, ուստի այս ճյուղը ավարտվում է:
- **traverse(5)** : 5 գազաթն ունի 20 արժեք; այն S -ին ավելացնելուց հետո $S = [0, 10, 20]$: 5 գազաթն ունի մեկ զբաղեցված գանգակ 2 համարով, հետևաբար

$c[5] = [2]$, և ֆունկցիան կանչում է `traverse(2)` :

- **`traverse(2)`** : 2 գազաթն ունի 30 արժեք; այն S -ին ավելացնելուց հետո $S = [0, 10, 20, 30]$: 2 գազաթն զբաղված գավակներ չունի, ուստի այս ճյուղը ավարտվում է:
- Կատարումը վերադառնում է `traverse(5)` ին, որը մշակել է $c[5]$ գազաթները, սա ավարտվում է:
- **`traverse(4)`** : 4 գազաթն ունի 20 արժեք; այն S -ին կցելը տալիս է $S = [0, 10, 20, 30, 20]$: 4 գազաթն զբաղված գավակներ չունի, ուստի այս կանչն ավարտվում է:

Բոլոր գանգերն ավարտվել են, և մշակման ենթակա հանգույցներ այլևս չկան: Վերջապես, մեքենայից հանվում են բոլոր գնդակները, և `collect` վերադարձնում է $S = [0, 10, 20, 30, 20]$ գանգվածը:

Ձեր խնդիրն է գտնել N գազաթների քանակը և ծնողների $R = [R[0], R[1], \dots, R[N - 2]]$ գանգվածը, որը նկարագրում է մեքենայի կառուցվածքը, բայց գազաթների հնարավոր տարբեր համարակալմամբ, այն գազաթների համար, որոնք տերևներ չեն և արմատը չեն: Ֆորմալ, ներկայացված R գանգվածը համարվում է ճիշտ, եթե հնարավոր է մեքենայի յուրաքանչյուր u գազաթին վերագրել եզակի $L[u]$ ՝ $0 \leq L[u] < N$ արժեքով, այնպես որ՝

- $L[u] = u$ բոլոր $0 \leq u < M$ -ի և $u = N - 1$ համար, և
- $L[P[u]] = R[L[u]]$ բոլոր $0 \leq u < N - 1$ համար:

$R[u] > u$ պարտադիր չէ: Լուծումը ներկայացնելիս մեքենան պետք է դատարկ լինի:

Ենթադրենք K -ն կատարված `collect` գործողությունների քանակն է, իսկ B -ն՝ բոլոր `insert` կանչերի դեպքում ցանկացած գնդակի վրա գրված առավելագույն արժեքը: Ենթադրենք $C = K + B$: Այնուհետև C **չպետք է գերազանցի 1000** : Որոշ ենթաառաջադրանքներում ձեր միավորը կախված է C -ի արժեքից:

Իրականացման մանրամասներ

Դուք պետք է իրականացնեք հետևյալ ֆունկցիան:

```
std::vector<int> find_structure(int M)
```

- M : մեքենայում տերևների քանակը:
- Ֆունկցիան կանչվում է յուրաքանչյուր թեստի համար ճիշտ մեկ անգամ:
- Ֆունկցիան պետք է վերադարձնի $R = [R[0], R[1], \dots, R[N - 2]]$ գանգված՝ $N - 1$ երկարությամբ, որը ծնողների ճիշտ գանգված է:

Մեքենայի հետ փոխազդելու համար ձեր ֆունկցիան կարող է կանչել հետևյալ երկու ֆունկցիաները:

Առաջին ֆունկցիան հետևյալն է.

```
bool insert(int U, int X)
```

- U : այն տերևի համարն է, որտեղ պետք է տեղադրվի գնդակը: Պետք է բավարարվի $0 \leq U < M$ պայմանը:
- X ՝ գնդակի ամբողջ թիվն է՝ արժեքը: Պետք է բավարարվի $0 \leq X \leq 1000$ պայմանը:
- Այս ֆունկցիան վերադարձնում է `true` եթե գնդակը հաջողությամբ է տեղադրվել, կամ `false` եթե U տերևն արդեն զբաղված է եղել:
- Այս ֆունկցիան կարող է կանչվել առավելագույնը 500 000 անգամ:

Երկրորդ ֆունկցիան հետևյալն է.

```
std::vector<int> collect()
```

- Հավաքում է բոլոր տեղադրված գնդակների արժեքները, դատարկում է մեքենան և վերադարձնում հավաքված S զանգվածը:

Եթե ձեր ծրագրի կատարման որևէ պահի C ի արժեքը գերազանցի 1000 -ը, ձեր լուծումը կստանա `Output isn't correct: Too many resources used:`

Մեքենայի կառուցվածքը ֆիքսված է `find_structure` ֆունկցիան կանչելուց առաջ: Գրեյդերը դետերմինացված է այն իմաստով, որ եթե այն գործարկեք երկու անգամ, և երկու գործարկումներն էլ կատարեն գործողությունների նույն հաջորդականությունը, ապա `collect`-ի կանչերը կվերադարձնեն նույն զանգվածները:

Սահմանափակումներ

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Ենթախնդիրներ

Ենթախնդիր	Գնահատական	Լրացուցիչ սահմանափակումներ
1	5	Արմատն ունի ճիշտ M զավակներ:
2	10	$M \leq 3$
3	25	$N \leq 200$, $M \leq 45$
4	60	Լրացուցիչ սահմանափակումներ չկան:

3-րդ և 4-րդ ենթաառաջադրանքներում ձեր միավորը կախված է C -ի արժեքից՝ հետևյալ կերպ:

Ենթախնդիր 3 (25 միավոր)

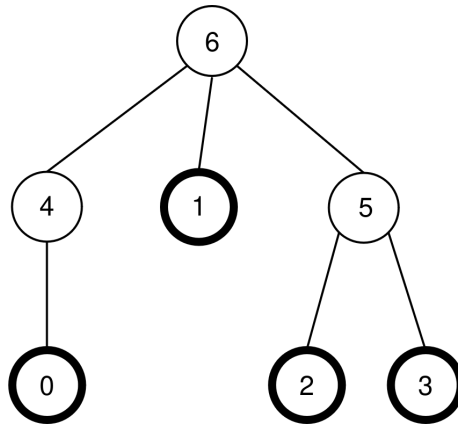
Սահմանաչափեր	Միավոր
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Ենթախնդիր 4 (60 միավոր)

Սահմանաչափեր	Միավոր
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Օրինակ

Դիտարկենք նույն մեքենան, ինչ խնդրի պահանջի մեջ, ուստի $N = 7$, $M = 4$, և $P = [4, 6, 5, 5, 6, 6]$: Մեքենան կրկին պատկերված է հետևյալ նկարում:



Գրեյդերը կատարում է հետևյալ գանգը.

```
find_structure(4)
```

Ֆունկցիան կարող է կատարել կանչերը հետևյալ հաջորդականությամբ.

1. **insert(0, 0)** : 0 տերևը ազատ է, ուստի 0 արժեքով գնդակը տեղադրվում է 0 տերևի վրա: $P[0] = 4$ գազաթը ազատ է, ուստի գնդակը տեղափոխվում է 4 համարով գազաթ: $P[4] = 6$ գազաթը ազատ է, ուստի գնդակը տեղափոխվում է 6 գազաթ: Քանի որ 6 գազաթը արմատն է, շարժումը կանգ է առնում: Կանչը վերադարձնում է `true` :
2. **insert(3, 20)** : 3 տերևը ազատ է, ուստի 20 արժեքով գնդակը տեղադրվում է 3 տերևի վրա: $P[3] = 5$ գազաթն ազատ է, ուստի գնդակը տեղափոխվում է 5 գազաթ: Քանի որ $P[5] = 6$ զբաղված է, շարժումը կանգ է առնում: Կանչը վերադարձնում է `true` :
3. **insert(1, 10)** : 1 տերևը ազատ է, ուստի 10 արժեքով գնդակը տեղադրվում է 1 տերևի վրա: Քանի որ $P[1] = 6$ զբաղեցված է, գնդակը մնում է 1 գազաթում: Կանչը վերադարձնում է `true` :
4. **insert(2, 30)** : 2 տերևը ազատ է, ուստի 30 արժեքով գնդակը տեղադրվում է 2 տերևի վրա: Քանի որ $P[2] = 5$ զբաղեցված է, գնդակը մնում է 2 գազաթում: Կանչը վերադարձնում է `true` :
5. **insert(1, 25)** : 1 տերևը զբաղեցված է, ուստի գնդակը չի տեղադրվում 1 տերևի վրա: Կանչը վերադարձնում է `false` :
6. **insert(0, 20)** : 0 տերևն ազատ է, ուստի 20 արժեքով գնդակը տեղադրվում է 0 տերևի վրա: $P[0] = 4$ գազաթն ազատ է, ուստի գնդակը տեղափոխվում է 4 գազաթ: Քանի որ $P[4] = 6$ զբաղված է, շարժումը դադարում է: Կանչը վերադարձնում է `true` :

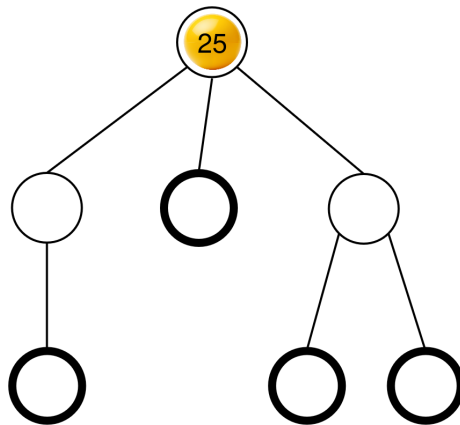
Գնդակների արդյունքում ստացված կոնֆիգուրացիան արդեն ցուցադրված էր խնդրի պահանջի մեջ:

Հաջորդը, ընթացակարգը կարող է կանչել՝

```
collect()
```

Այս կանչը վերադարձնում է $S = [0, 10, 20, 30, 20]$: Դրանից հետո մեքենան կրկին դատարկ է:

Հաջորդը, պրոցեսորան կարող է կանչել `insert(2,25)` : 2 տերևը ազատ է, ուստի 25 արժեքով գնդակը տեղադրվում է 2 տերևի վրա: Այնուհետև այս գնդակը տեղափոխվում է 5 գագաթ, ապա՝ 6 գագաթ, որտեղ էլ կանգ է առնում: Կանչը վերադարձնում է `true` : Գնդակների արդյունքում ստացված կոնֆիգուրացիան ցույց է տրված հետևյալ նկարում:



Վերջապես, պրոցեսորան կարող է կանչել `collect()` , որը վերադարձնում է $S = [25]$ և դատարկում է մեքենան:

Կան երկու ճիշտ ծնողական զանգվածներ, որոնք `find_structure` կարող է վերադարձնել՝ $R = P = [4, 6, 5, 5, 6, 6]$ (որը համապատասխանում է $L = [0, 1, 2, 3, 4, 5, 6]$ համարակալմանը) և $R = [5, 6, 4, 4, 6, 6]$ (որը համապատասխանում է $L = [0, 1, 2, 3, 5, 4, 6]$ համարակալմանը): Այս օրինակում $K = 2$ և $B = 30$, ուստի $C = 32$:

Նմուշային գրեյդեր

Մուտքի ձևաչափը՝

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Ելքի ձևաչափը՝

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Այստեղ Q -ն `find_structure` ի կողմից վերադարձված R զանգվածի երկարությունն է: